

Сервер аутентификации и обработки данных

Создано системой Doxygen 1.9.4

1	Алфавитный указатель классов	1
1.1	Классы	1
2	Список файлов	3
2.1	Файлы	3
3	Классы	5
3.1	Класс Authenticator	5
3.1.1	Подробное описание	5
3.1.2	Методы	5
3.1.2.1	generateSalt()	6
3.1.2.2	hashPassword()	6
3.1.2.3	toHexString()	6
3.1.2.4	verifyPassword()	7
3.2	Класс ClientDatabase	8
3.2.1	Подробное описание	8
3.2.2	Методы	8
3.2.2.1	getPassword()	8
3.2.2.2	loadFromFile()	9
3.2.2.3	userExists()	9
3.3	Класс Configuration	10
3.3.1	Подробное описание	10
3.3.2	Методы	11
3.3.2.1	getClientDBFile()	11
3.3.2.2	getLogFile()	11
3.3.2.3	getPort()	11
3.3.2.4	parseCommandLine()	11
3.4	Класс ConnectionHandler	12
3.4.1	Подробное описание	13
3.4.2	Конструктор(ы)	14
3.4.2.1	ConnectionHandler()	14
3.4.3	Методы	14
3.4.3.1	authenticateClient()	14
3.4.3.2	handleConnection()	15
3.4.3.3	processData()	15
3.4.3.4	receiveExactly()	15
3.4.3.5	receiveWithTimeout()	16
3.4.3.6	recvWithTimeout()	16
3.4.3.7	sendExactly()	18
3.4.3.8	waitForData()	18
3.5	Класс DataProcessor	19
3.5.1	Подробное описание	19
3.5.2	Методы	19
3.5.2.1	calculateAverage()	19

3.5.2.2 <code>handleOverflow()</code>	20
3.6 Класс <code>Logger</code>	20
3.6.1 Подробное описание	21
3.6.2 Методы	21
3.6.2.1 <code>getCurrentTime()</code>	21
3.6.2.2 <code>initialize()</code>	21
3.6.2.3 <code>logError()</code>	22
3.6.2.4 <code>logInfo()</code>	22
3.7 Класс <code>Server</code>	23
3.7.1 Подробное описание	24
3.7.2 Конструктор(ы)	24
3.7.2.1 <code>~Server()</code>	24
3.7.3 Методы	24
3.7.3.1 <code>acceptConnections()</code>	24
3.7.3.2 <code>initializeSocket()</code>	25
3.7.3.3 <code>start()</code>	25
3.7.3.4 <code>stop()</code>	26
4 Файлы	27
4.1 Файл <code>Authenticator.cpp</code>	27
4.1.1 Подробное описание	27
4.2 Файл <code>Authenticator.h</code>	27
4.2.1 Подробное описание	28
4.3 <code>Authenticator.h</code>	29
4.4 Файл <code>ClientDatabase.cpp</code>	29
4.4.1 Подробное описание	29
4.5 Файл <code>ClientDatabase.h</code>	30
4.5.1 Подробное описание	30
4.6 <code>ClientDatabase.h</code>	31
4.7 Файл <code>Configuration.cpp</code>	31
4.7.1 Подробное описание	31
4.8 Файл <code>Configuration.h</code>	31
4.8.1 Подробное описание	32
4.9 <code>Configuration.h</code>	33
4.10 Файл <code>ConnectionHandler.cpp</code>	33
4.10.1 Подробное описание	33
4.11 Файл <code>ConnectionHandler.h</code>	33
4.11.1 Подробное описание	34
4.12 <code>ConnectionHandler.h</code>	35
4.13 Файл <code>DataProcessor.cpp</code>	35
4.13.1 Подробное описание	36
4.14 Файл <code>DataProcessor.h</code>	36
4.14.1 Подробное описание	37

4.15 DataProcessor.h	37
4.16 Файл Logger.cpp	38
4.16.1 Подробное описание	38
4.17 Файл Logger.h	38
4.17.1 Подробное описание	39
4.18 Logger.h	39
4.19 Файл main.cpp	40
4.19.1 Подробное описание	40
4.19.2 Функции	40
4.19.2.1 main()	40
4.19.2.2 signalHandler()	41
4.20 Файл Server.cpp	41
4.20.1 Подробное описание	42
4.21 Файл Server.h	42
4.21.1 Подробное описание	43
4.22 Server.h	43
Предметный указатель	45

Глава 1

Алфавитный указатель классов

1.1 Классы

Классы с их кратким описанием.

Authenticator	Класс для работы с аутентификацией пользователей	5
ClientDatabase	Класс для работы с базой данных пользователей	8
Configuration	Класс для управления конфигурацией сервера	10
ConnectionHandler	Класс для обработки клиентских соединений	12
DataProcessor	Класс для обработки числовых данных	19
Logger	Класс для ведения журнала событий	20
Server	Основной класс сервера приложения	23

Глава 2

Список файлов

2.1 Файлы

Полный список документированных файлов.

Authenticator.cpp	
Реализация класса Authenticator	27
Authenticator.h	
Заголовочный файл класса Authenticator для аутентификации пользователей . . .	27
ClientDatabase.cpp	
Реализация класса ClientDatabase	29
ClientDatabase.h	
Заголовочный файл класса ClientDatabase для работы с базой пользователей . . .	30
Configuration.cpp	
Реализация класса Configuration	31
Configuration.h	
Заголовочный файл класса Configuration для управления настройками сервера .	31
ConnectionHandler.cpp	
Реализация класса ConnectionHandler	33
ConnectionHandler.h	
Заголовочный файл класса ConnectionHandler для обработки клиентских соедине- ний	33
DataProcessor.cpp	
Реализация класса DataProcessor	35
DataProcessor.h	
Заголовочный файл класса DataProcessor для обработки данных	36
Logger.cpp	
Реализация класса Logger	38
Logger.h	
Заголовочный файл класса Logger для ведения журнала событий	38
main.cpp	
Основной файл приложения сервера	40
Server.cpp	
Реализация класса Server	41
Server.h	
Заголовочный файл класса Server для управления сервером	42

Глава 3

Классы

3.1 Класс Authenticator

Класс для работы с аутентификацией пользователей

```
#include <Authenticator.h>
```

Открытые статические члены

- static std::string [generateSalt](#) ()
Генерирует случайную соль для хеширования пароля
- static std::string [hashPassword](#) (const std::string &salt, const std::string &password)
Хеширует пароль с использованием соли
- static bool [verifyPassword](#) (const std::string &passwordHash, const std::string &salt, const std::string &storedPassword)
Проверяет корректность пароля
- static std::string [toHexString](#) (const std::string &binary)
Преобразует бинарные данные в шестнадцатеричную строку

Закрытые статические данные

- static constexpr int SALT_SIZE = 8
Размер соли в байтах (64 бита)

3.1.1 Подробное описание

Класс для работы с аутентификацией пользователей

Предоставляет статические методы для генерации соли, хеширования паролей и проверки аутентификационных данных с использованием алгоритма MD5.

3.1.2 Методы

3.1.2.1 generateSalt()

```
std::string Authenticator::generateSalt ( ) [static]
```

Генерирует случайную соль для хеширования пароля

Возвращает

Строка с шестнадцатеричным представлением соли

Строка с шестнадцатеричным представлением соли

Использует криптографически безопасный генератор случайных чисел для создания 8-байтовой (64-битной) соли.

3.1.2.2 hashPassword()

```
std::string Authenticator::hashPassword (
    const std::string & salt,
    const std::string & password ) [static]
```

Хеширует пароль с использованием соли

Аргументы

salt	Соль для хеширования
password	Пароль пользователя

Возвращает

Хеш пароля в шестнадцатеричном формате

Аргументы

salt	Соль для хеширования
password	Пароль пользователя

Возвращает

Хеш пароля в шестнадцатеричном формате

Использует алгоритм MD5 для создания хеша из конкатенации соли и пароля. Результат возвращается в виде шестнадцатеричной строки.

3.1.2.3 toHexString()

```
std::string Authenticator::toHexString (
    const std::string & binary ) [static]
```

Преобразует бинарные данные в шестнадцатеричную строку

Аргументы

binary	Бинарные данные
--------	-----------------

Возвращает

Шестнадцатеричное представление данных

Аргументы

binary	Бинарные данные
--------	-----------------

Возвращает

Шестнадцатеричное представление данных

Каждый байт преобразуется в два шестнадцатеричных символа. Результат дополняется слева нулями до 16 символов.

3.1.2.4 verifyPassword()

```
bool Authenticator::verifyPassword (
    const std::string & passwordHash,
    const std::string & salt,
    const std::string & storedPassword ) [static]
```

Проверяет корректность пароля

Аргументы

passwordHash	Хеш пароля для проверки
salt	Соль, использованная при хешировании
storedPassword	Оригинальный пароль из базы данных

Возвращает

true если пароль верный, false в противном случае

Аргументы

passwordHash	Хеш пароля для проверки
salt	Соль, использованная при хешировании
storedPassword	Оригинальный пароль из базы данных

Возвращает

true если пароль верный, false в противном случае

Вычисляет хеш от хранимого пароля с переданной солью и сравнивает с предоставленным хешем. Сравнение выполняется без учета регистра.

Объявления и описания членов классов находятся в файлах:

- [Authenticator.h](#)
- [Authenticator.cpp](#)

3.2 Класс ClientDatabase

Класс для работы с базой данных пользователей

```
#include <ClientDatabase.h>
```

Открытые члены

- bool [loadFromFile](#) (const std::string &[filename](#))
Загружает базу пользователей из файла
- bool [userExists](#) (const std::string &login) const
Проверяет существование пользователя в базе
- std::string [getPassword](#) (const std::string &login) const
Получает пароль пользователя

Закрытые данные

- std::unordered_map< std::string, std::string > users
Хэш-таблица: логин -> пароль
- std::string filename
Имя файла базы данных

3.2.1 Подробное описание

Класс для работы с базой данных пользователей

Предоставляет методы для загрузки базы пользователей из файла, проверки существования пользователя и получения пароля.

3.2.2 Методы

3.2.2.1 getPassword()

```
std::string ClientDatabase::getPassword (  
    const std::string & login ) const
```

Получает пароль пользователя

Аргументы

login	Логин пользователя
-------	--------------------

Возвращает

Пароль пользователя или пустая строка если пользователь не найден

3.2.2.2 loadFromFile()

```
bool ClientDatabase::loadFromFile (  
    const std::string & fname )
```

Загружает базу пользователей из файла

Аргументы

filename	Имя файла базы данных
----------	-----------------------

Возвращает

true если загрузка успешна, false в противном случае

Аргументы

fname	Имя файла базы данных
-------	-----------------------

Возвращает

true если загрузка успешна, false в противном случае

Формат файла: каждая строка содержит логин и пароль, разделенные двоеточием. Пример: user1↵:password1

3.2.2.3 userExists()

```
bool ClientDatabase::userExists (  
    const std::string & login ) const
```

Проверяет существование пользователя в базе

Аргументы

login	Логин пользователя
-------	--------------------

Возвращает

true если пользователь существует, false в противном случае

Объявления и описания членов классов находятся в файлах:

- [ClientDatabase.h](#)
- [ClientDatabase.cpp](#)

3.3 Класс Configuration

Класс для управления конфигурацией сервера

```
#include <Configuration.h>
```

Открытые члены

- bool [parseCommandLine](#) (int argc, char *argv[])
Парсит параметры командной строки
- const std::string & [getClientDBFile](#) () const
Получает путь к файлу базы клиентов
- const std::string & [getLogFile](#) () const
Получает путь к файлу журнала
- int [getPort](#) () const
Получает номер порта сервера

Открытые статические члены

- static void [printHelp](#) ()
Выводит справку по использованию программы

Закрытые данные

- std::string clientDBFile = "./etc/vcalc.conf"
Файл базы клиентов
- std::string logFile = "./var/log/vcalc.log"
Файл журнала
- int port = 33333
Порт сервера

3.3.1 Подробное описание

Класс для управления конфигурацией сервера

Предоставляет методы для парсинга параметров командной строки и хранения настроек сервера.

3.3.2 Методы

3.3.2.1 getClientDBFile()

```
const std::string & Configuration::getClientDBFile ( ) const [inline]
```

Получает путь к файлу базы клиентов

Возвращает

Путь к файлу базы клиентов

3.3.2.2 getLogFile()

```
const std::string & Configuration::getLogFile ( ) const [inline]
```

Получает путь к файлу журнала

Возвращает

Путь к файлу журнала

3.3.2.3 getPort()

```
int Configuration::getPort ( ) const [inline]
```

Получает номер порта сервера

Возвращает

Номер порта

3.3.2.4 parseCommandLine()

```
bool Configuration::parseCommandLine (
    int argc,
    char * argv[] )
```

Парсит параметры командной строки

Аргументы

argc	Количество аргументов
argv	Массив аргументов

Возвращает

true если парсинг успешен, false в противном случае

Аргументы

argc	Количество аргументов
argv	Массив аргументов

Возвращает

true если парсинг успешен, false в противном случае

Поддерживаемые параметры: -h, -help Показать справку -c, -config FILE Файл базы клиентов -l, -log FILE Файл журнала -p, -port PORT Порт сервера

Объявления и описания членов классов находятся в файлах:

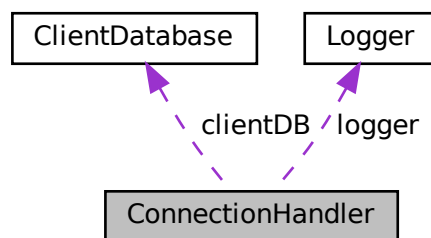
- [Configuration.h](#)
- [Configuration.cpp](#)

3.4 Класс ConnectionHandler

Класс для обработки клиентских соединений

```
#include <ConnectionHandler.h>
```

Граф связей класса ConnectionHandler:



Открытые члены

- `ConnectionHandler` (int socket, `ClientDatabase` &db, `Logger` &log, const sockaddr_in &addr)
Конструктор класса `ConnectionHandler`.
- void `handleConnection` ()
Обрабатывает клиентское соединение

Закрытые члены

- bool `authenticateClient` ()
Выполняет аутентификацию клиента
- bool `processData` ()
Обрабатывает данные от клиента
- bool `receiveExactly` (void *buffer, size_t size)
Принимает точное количество байт
- bool `sendExactly` (const void *buffer, size_t size)
Отправляет точное количество байт
- bool `waitForData` (int timeoutSeconds=`TIMEOUT_SECONDS`)
Ожидает доступности данных для чтения
- bool `receiveWithTimeout` (void *buffer, size_t size, int timeoutSeconds=`TIMEOUT_SECONDS`)
Принимает данные с таймаутом
- ssize_t `recvWithTimeout` (void *buffer, size_t size, int timeoutSeconds=`TIMEOUT_SECONDS`)
Принимает данные с таймаутом (низкоуровневый метод)

Закрытые данные

- int clientSocket
Сокет клиентского соединения
- `ClientDatabase` & clientDB
Ссылка на базу данных пользователей
- `Logger` & logger
Ссылка на логгер
- sockaddr_in clientAddr
Адрес клиента

Закрытые статические данные

- static constexpr int TIMEOUT_SECONDS = 5
Таймаут операций в секундах

3.4.1 Подробное описание

Класс для обработки клиентских соединений

Обрабатывает отдельное клиентское соединение, включая аутентификацию, прием и обработку данных, отправку результатов.

3.4.2 Конструктор(ы)

3.4.2.1 ConnectionHandler()

```
ConnectionHandler::ConnectionHandler (
    int socket,
    ClientDatabase & db,
    Logger & log,
    const sockaddr_in & addr )
```

Конструктор класса [ConnectionHandler](#).

Аргументы

socket	Сокет клиентского соединения
db	Ссылка на базу данных пользователей
log	Ссылка на логгер
addr	Адрес клиента

3.4.3 Методы

3.4.3.1 authenticateClient()

```
bool ConnectionHandler::authenticateClient ( ) [private]
```

Выполняет аутентификацию клиента

Возвращает

true если аутентификация успешна, false в противном случае
true если аутентификация успешна, false в противном случае

Процесс аутентификации:

1. Получение логина от клиента
2. Генерация и отправка соли
3. Получение хеша пароля
4. Проверка хеша с использованием базы данных

3.4.3.2 handleConnection()

```
void ConnectionHandler::handleConnection ( )
```

Обрабатывает клиентское соединение

Выполняет аутентификацию и обработку данных, затем закрывает соединение.

Основной метод обработки соединения:

1. Установка таймаутов на сокет
2. Аутентификация клиента
3. Обработка данных
4. Закрытие соединения

3.4.3.3 processData()

```
bool ConnectionHandler::processData ( ) [private]
```

Обрабатывает данные от клиента

Возвращает

true если обработка успешна, false в противном случае
true если обработка успешна, false в противном случае

Процесс обработки данных:

1. Получение количества векторов
2. Для каждого вектора:
 - Получение размера вектора
 - Получение значений вектора
 - Вычисление среднего значения
 - Отправка результата

3.4.3.4 receiveExactly()

```
bool ConnectionHandler::receiveExactly (
    void * buffer,
    size_t size ) [private]
```

Принимает точное количество байт

Аргументы

buffer	Буфер для приема данных
size	Количество байт для приема

Возвращает

true если прием успешен, false в противном случае

3.4.3.5 receiveWithTimeout()

```
bool ConnectionHandler::receiveWithTimeout (  
    void * buffer,  
    size_t size,  
    int timeoutSeconds = TIMEOUT\_SECONDS ) [private]
```

Принимает данные с таймаутом

Аргументы

buffer	Буфер для приема данных
size	Количество байт для приема
timeoutSeconds	Таймаут в секундах

Возвращает

true если прием успешен, false в противном случае

Аргументы

buffer	Буфер для приема данных
size	Количество байт для приема
timeoutSeconds	Таймаут в секундах

Возвращает

true если прием успешен, false в противном случае

Гарантированно принимает указанное количество байт или возвращает ошибку.

3.4.3.6 recvWithTimeout()

```
ssize_t ConnectionHandler::recvWithTimeout (  
    void * buffer,
```

```
size_t size,  
int timeoutSeconds = TIMEOUT_SECONDS ) [private]
```

Принимает данные с таймаутом (низкоуровневый метод)

Аргументы

buffer	Буфер для приема данных
size	Максимальный размер буфера
timeoutSeconds	Таймаут в секундах

Возвращает

Количество принятых байт или -1 при ошибке

3.4.3.7 sendExactly()

```
bool ConnectionHandler::sendExactly (
    const void * buffer,
    size_t size ) [private]
```

Отправляет точное количество байт

Аргументы

buffer	Буфер с данными для отправки
size	Количество байт для отправки

Возвращает

true если отправка успешна, false в противном случае

Аргументы

buffer	Буфер с данными для отправки
size	Количество байт для отправки

Возвращает

true если отправка успешна, false в противном случае

Гарантированно отправляет указанное количество байт или возвращает ошибку.

3.4.3.8 waitForData()

```
bool ConnectionHandler::waitForData (
    int timeoutSeconds = TIMEOUT_SECONDS ) [private]
```

Ожидает доступности данных для чтения

Аргументы

timeoutSeconds	Таймаут в секундах
----------------	--------------------

Возвращает

true если данные доступны, false при таймауте или ошибке

Аргументы

timeoutSeconds	Таймаут в секундах
----------------	--------------------

Возвращает

true если данные доступны, false при таймауте или ошибке

Использует системный вызов select для ожидания данных с таймаутом.

Объявления и описания членов классов находятся в файлах:

- [ConnectionHandler.h](#)
- [ConnectionHandler.cpp](#)

3.5 Класс DataProcessor

Класс для обработки числовых данных

```
#include <DataProcessor.h>
```

Открытые статические члены

- static uint32_t [calculateAverage](#) (const std::vector< uint32_t > &vector)
Вычисляет среднее значение элементов вектора
- static uint32_t [handleOverflow](#) (uint64_t value)
Обработывает переполнение при преобразовании типов

3.5.1 Подробное описание

Класс для обработки числовых данных

Предоставляет статические методы для вычисления среднего значения вектора чисел и обработки переполнения при преобразовании типов.

3.5.2 Методы

3.5.2.1 calculateAverage()

```
uint32_t DataProcessor::calculateAverage (
    const std::vector< uint32_t > & vector ) [static]
```

Вычисляет среднее значение элементов вектора

Аргументы

vector	Вектор значений
--------	-----------------

Возвращает

Среднее значение вектора или 0 если вектор пуст

Аргументы

vector	Вектор значений
--------	-----------------

Возвращает

Среднее значение вектора или 0 если вектор пуст

Вычисляет сумму всех элементов и делит на количество элементов. Использует 64-битную арифметику для избежания переполнения.

3.5.2.2 handleOverflow()

```
uint32_t DataProcessor::handleOverflow (
    uint64_t value ) [static]
```

Обрабатывает переполнение при преобразовании типов

Аргументы

value	Значение для преобразования
-------	-----------------------------

Возвращает

Значение, ограниченное диапазоном uint32_t

Если значение больше UINT32_MAX, возвращает UINT32_MAX. Если значение меньше 0, возвращает 0. Иначе возвращает значение, приведенное к uint32_t.

Объявления и описания членов классов находятся в файлах:

- [DataProcessor.h](#)
- [DataProcessor.cpp](#)

3.6 Класс Logger

Класс для ведения журнала событий

```
#include <Logger.h>
```

Открытые члены

- `bool initialize (const std::string &filename)`
Инициализирует логгер
- `void close ()`
Закрывает логгер
- `void logError (const std::string &message, bool critical=false)`
Записывает сообщение об ошибке
- `void logInfo (const std::string &message)`
Записывает информационное сообщение

Закрытые члены

- `std::string getCurrentTime ()`
Получает текущее время в формате строки

Закрытые данные

- `std::ofstream logFile`
Поток для записи в файл журнала
- `std::mutex logMutex`
Мьютекс для синхронизации доступа
- `std::string filename`
Имя файла журнала

3.6.1 Подробное описание

Класс для ведения журнала событий

Предоставляет потокобезопасные методы для записи сообщений разных уровней важности в файл и на консоль.

3.6.2 Методы

3.6.2.1 `getCurrentTime()`

```
std::string Logger::getCurrentTime ( ) [private]
```

Получает текущее время в формате строки

Возвращает

Строка с текущим временем в формате "YYYY-MM-DD HH:MM:SS"

3.6.2.2 `initialize()`

```
bool Logger::initialize (
    const std::string & fname )
```

Инициализирует логгер

Аргументы

filename	Имя файла для записи журнала
----------	------------------------------

Возвращает

true если инициализация успешна, false в противном случае

Аргументы

fname	Имя файла для записи журнала
-------	------------------------------

Возвращает

true если инициализация успешна, false в противном случае

3.6.2.3 logError()

```
void Logger::logError (
    const std::string & message,
    bool critical = false )
```

Записывает сообщение об ошибке

Аргументы

message	Текст сообщения
critical	Флаг критичности ошибки (true для критических ошибок)
message	Текст сообщения
critical	Флаг критичности ошибки (true для критических ошибок)

Записывает сообщение в файл журнала и выводит на stderr. Критические ошибки помечаются меткой "CRITICAL", обычные - "ERROR".

3.6.2.4 logInfo()

```
void Logger::logInfo (
    const std::string & message )
```

Записывает информационное сообщение

Аргументы

message	Текст сообщения
message	Текст сообщения

Записывает сообщение в файл журнала и выводит на stdout.

Объявления и описания членов классов находятся в файлах:

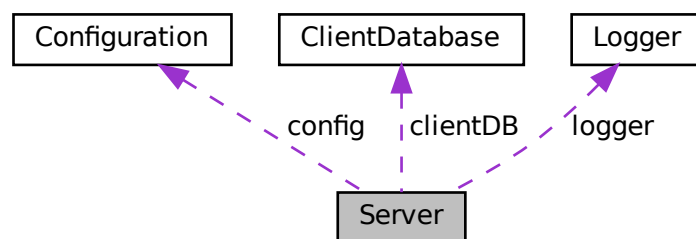
- [Logger.h](#)
- [Logger.cpp](#)

3.7 Класс Server

Основной класс сервера приложения

```
#include <Server.h>
```

Граф связей класса Server:



Открытые члены

- `Server ()`
Конструктор класса `Server`.
- `~Server ()`
Деструктор класса `Server`.
- `bool start (int argc, char *argv[])`
Запускает сервер
- `void stop ()`
Останавливает сервер

Закрытые члены

- `bool initializeSocket ()`
Инициализирует сокет сервера
- `void acceptConnections ()`
Принимает входящие соединения

Закрытые данные

- [Configuration](#) config
Конфигурация сервера
- [ClientDatabase](#) clientDB
База данных пользователей
- [Logger](#) logger
Логгер для записи событий
- `std::atomic< bool > running`
Флаг работы сервера
- `int serverSocket`
Основной сокет сервера

3.7.1 Подробное описание

Основной класс сервера приложения

Управляет инициализацией, запуском и остановкой сервера, обработкой входящих соединений и координацией работы всех компонентов.

3.7.2 Конструктор(ы)

3.7.2.1 ~Server()

`Server::~Server ( )`

Деструктор класса [Server](#).

Автоматически останавливает сервер при уничтожении объекта.

3.7.3 Методы

3.7.3.1 acceptConnections()

`void Server::acceptConnections ( ) [private]`

Принимает входящие соединения

Запускает бесконечный цикл приема соединений, каждое соединение обрабатывается в отдельном потоке.

Запускает бесконечный цикл приема соединений, каждое соединение обрабатывается в отдельном потоке с помощью [ConnectionHandler](#).

3.7.3.2 initializeSocket()

```
bool Server::initializeSocket ( ) [private]
```

Инициализирует сокет сервера

Возвращает

true если инициализация успешна, false в противном случае
true если инициализация успешна, false в противном случае

Создает сокет, устанавливает опции, привязывает к порту и переводит в режим прослушивания.

3.7.3.3 start()

```
bool Server::start (
    int argc,
    char * argv[] )
```

Запускает сервер

Аргументы

argc	Количество аргументов командной строки
argv	Массив аргументов командной строки

Возвращает

true если запуск успешен, false в противном случае

Аргументы

argc	Количество аргументов командной строки
argv	Массив аргументов командной строки

Возвращает

true если запуск успешен, false в противном случае

Последовательность запуска:

1. Парсинг параметров командной строки
2. Инициализация логгера
3. Загрузка базы данных пользователей
4. Инициализация сокета
5. Запуск цикла приема соединений

3.7.3.4 stop()

```
void Server::stop ( )
```

Останавливает сервер

Устанавливает флаг `running` в `false`, закрывает сокет сервера и логгер. Поток `acceptConnections` автоматически завершится.

Объявления и описания членов классов находятся в файлах:

- [Server.h](#)
- [Server.cpp](#)

Глава 4

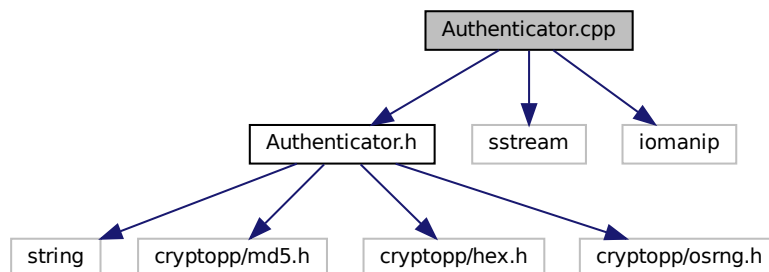
Файлы

4.1 Файл Authenticator.cpp

Реализация класса [Authenticator](#).

```
#include "Authenticator.h"  
#include <sstream>  
#include <iomanip>
```

Граф включаемых заголовочных файлов для Authenticator.cpp:



4.1.1 Подробное описание

Реализация класса [Authenticator](#).

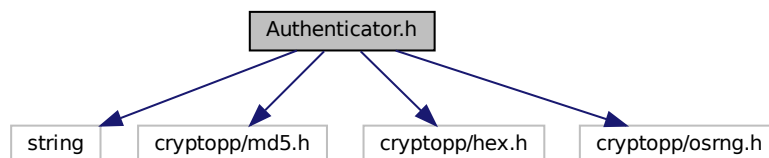
4.2 Файл Authenticator.h

Заголовочный файл класса [Authenticator](#) для аутентификации пользователей

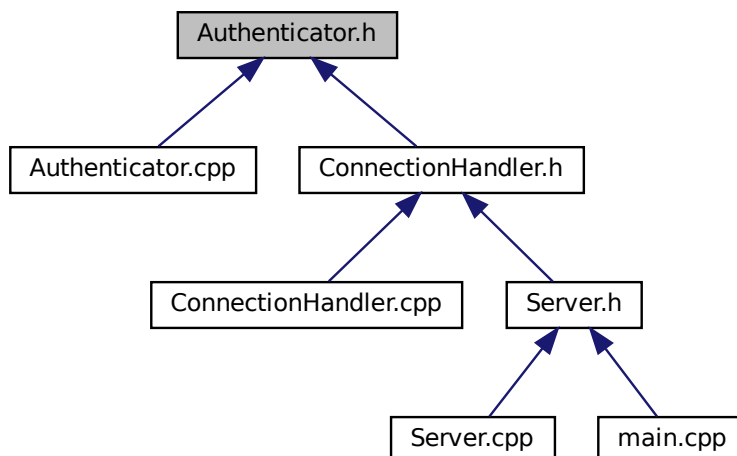
```
#include <string>  
#include <cryptopp/md5.h>  
#include <cryptopp/hex.h>
```

```
#include <cryptopp/osrng.h>
```

Граф включаемых заголовочных файлов для Authenticator.h:



Граф файлов, в которые включается этот файл:



Классы

- class [Authenticator](#)
Класс для работы с аутентификацией пользователей

Макросы

- `#define CRYPTOPP_ENABLE_NAMESPACE_WEAK 1`

4.2.1 Подробное описание

Заголовочный файл класса [Authenticator](#) для аутентификации пользователей

Содержит объявление класса [Authenticator](#), который отвечает за генерацию соли, хеширование паролей и проверку аутентификационных данных с использованием MD5.

4.3 Authenticator.h

См. документацию.

```

1
9 #pragma once
10
11 #include <string>
12
13 // Определяем для использования MD5 (как требуется в ТЗ)
14 #define CRYPTOPP_ENABLE_NAMESPACE_WEAK 1
15 #include <cryptopp/md5.h>
16 #include <cryptopp/hex.h>
17 #include <cryptopp/osrng.h>
18
26 class Authenticator {
27 private:
28     static constexpr int SALT_SIZE = 8;
29
30 public:
35     static std::string generateSalt();
36
43     static std::string hashPassword(const std::string& salt, const std::string& password);
44
52     static bool verifyPassword(const std::string& passwordHash,
53                               const std::string& salt, const std::string& storedPassword);
54
60     static std::string toHexString(const std::string& binary);
61 };

```

4.4 Файл ClientDatabase.cpp

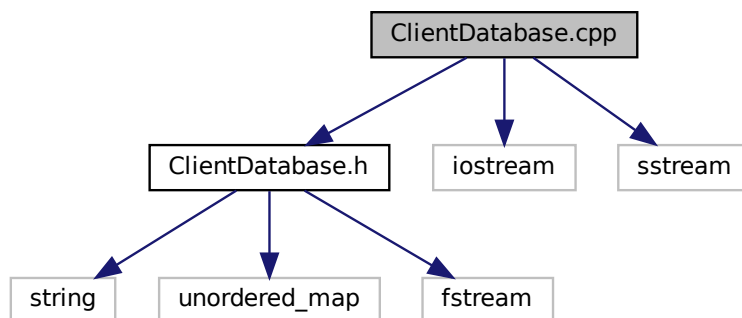
Реализация класса `ClientDatabase`.

```

#include "ClientDatabase.h"
#include <iostream>
#include <sstream>

```

Граф включаемых заголовочных файлов для `ClientDatabase.cpp`:



4.4.1 Подробное описание

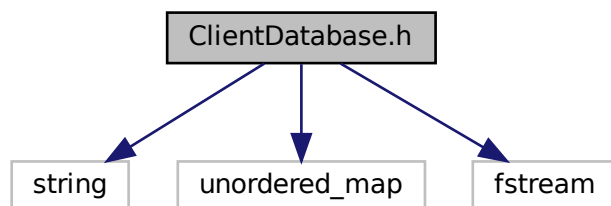
Реализация класса `ClientDatabase`.

4.5 Файл ClientDatabase.h

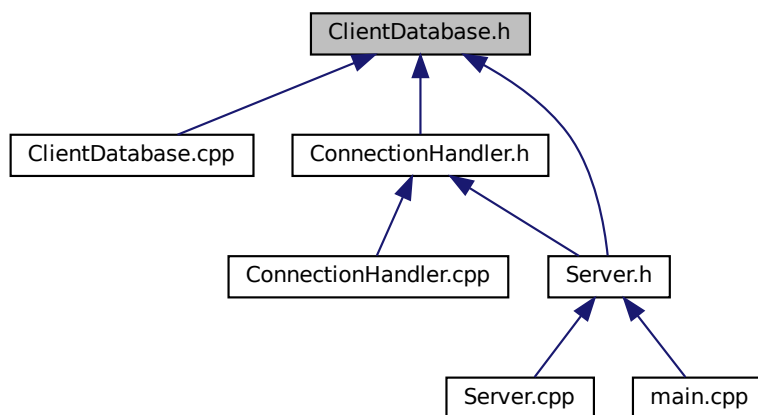
Заголовочный файл класса `ClientDatabase` для работы с базой пользователей

```
#include <string>
#include <unordered_map>
#include <fstream>
```

Граф включаемых заголовочных файлов для `ClientDatabase.h`:



Граф файлов, в которые включается этот файл:



Классы

- class `ClientDatabase`
Класс для работы с базой данных пользователей

4.5.1 Подробное описание

Заголовочный файл класса `ClientDatabase` для работы с базой пользователей

Содержит объявление класса `ClientDatabase`, который управляет загрузкой и доступом к базе данных пользователей.

4.6 ClientDatabase.h

См. документацию.

```

1
9 #pragma once
10
11 #include <string>
12 #include <unordered_map>
13 #include <fstream>
14
22 class ClientDatabase {
23 private:
24     std::unordered_map<std::string, std::string> users;
25     std::string filename;
26
27 public:
33     bool loadFromFile(const std::string& filename);
34
40     bool userExists(const std::string& login) const;
41
47     std::string getPassword(const std::string& login) const;
48 };

```

4.7 Файл Configuration.cpp

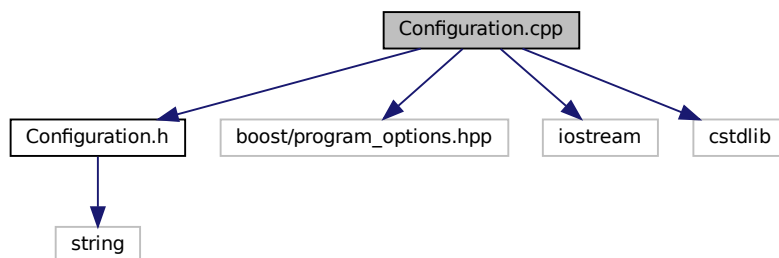
Реализация класса [Configuration](#).

```

#include "Configuration.h"
#include <boost/program_options.hpp>
#include <iostream>
#include <cstdlib>

```

Граф включаемых заголовочных файлов для Configuration.cpp:



4.7.1 Подробное описание

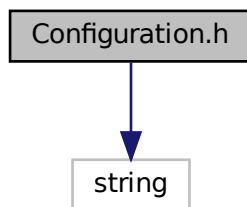
Реализация класса [Configuration](#).

4.8 Файл Configuration.h

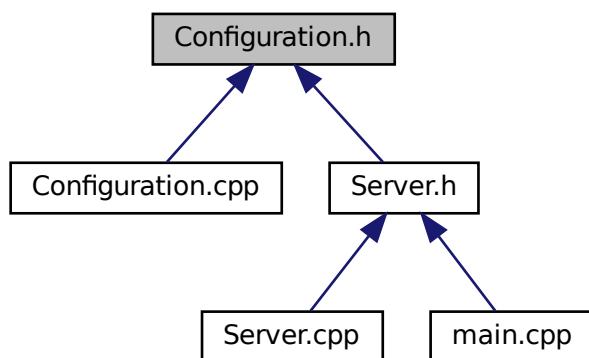
Заголовочный файл класса [Configuration](#) для управления настройками сервера

```
#include <string>
```

Граф включаемых заголовочных файлов для Configuration.h:



Граф файлов, в которые включается этот файл:



Классы

- class [Configuration](#)
Класс для управления конфигурацией сервера

4.8.1 Подробное описание

Заголовочный файл класса [Configuration](#) для управления настройками сервера

Содержит объявление класса [Configuration](#), который обрабатывает параметры командной строки и хранит конфигурацию сервера.

4.9 Configuration.h

См. документацию.

```

1
9 #pragma once
10
11 #include <string>
12
20 class Configuration {
21 private:
22     std::string clientDBFile = "./etc/vcalc.conf";
23     std::string logFile = "./var/log/vcalc.log";
24     int port = 33333;
25
26 public:
33     bool parseCommandLine(int argc, char* argv[]);
34
39     const std::string& getClientDBFile()const { return clientDBFile; }
40
45     const std::string& getLogFile()const { return logFile; }
46
51     int getPort()const { return port; }
52
56     static void printHelp();
57 };

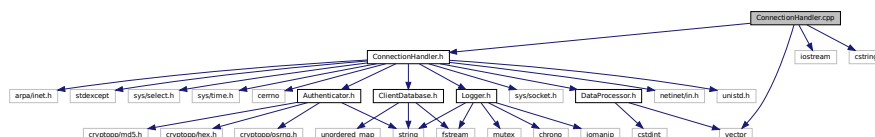
```

4.10 Файл ConnectionHandler.cpp

Реализация класса `ConnectionHandler`.

```
#include "ConnectionHandler.h"
#include <iostream>
#include <vector>
#include <cstring>
```

Граф включаемых заголовочных файлов для ConnectionHandler.cpp:



4.10.1 Подробное описание

Реализация класса `ConnectionHandler`.

4.11 Файл ConnectionHandler.h

Заголовочный файл класса `ConnectionHandler` для обработки клиентских соединений

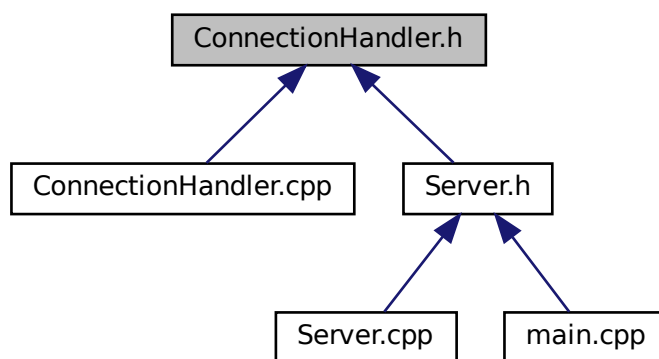
```
#include <sys/socket.h>
#include <netinet/in.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <stdexcept>
#include <sys/select.h>
```

```
#include <sys/time.h>
#include <cerrno>
#include "ClientDatabase.h"
#include "Authenticator.h"
#include "DataProcessor.h"
#include "Logger.h"
```

Граф включаемых заголовочных файлов для ConnectionHandler.h:



Граф файлов, в которые включается этот файл:



Классы

- class [ConnectionHandler](#)

Класс для обработки клиентских соединений

4.11.1 Подробное описание

Заголовочный файл класса [ConnectionHandler](#) для обработки клиентских соединений

Содержит объявление класса [ConnectionHandler](#), который обрабатывает отдельные клиентские соединения, включая аутентификацию и обработку данных.

4.12 ConnectionHandler.h

[См. документацию.](#)

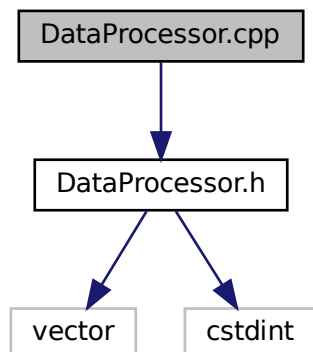
```
1
9 #pragma once
10
11 #include <sys/socket.h>
12 #include <netinet/in.h>
13 #include <unistd.h>
14 #include <arpa/inet.h>
15 #include <stdexcept>
16 #include <sys/select.h>
17 #include <sys/time.h>
18 #include <cerrno>
19 #include "ClientDatabase.h"
20 #include "Authenticator.h"
21 #include "DataProcessor.h"
22 #include "Logger.h"
23
31 class ConnectionHandler {
32 private:
33     int clientSocket;
34     ClientDatabase& clientDB;
35     Logger& logger;
36     sockaddr_in clientAddr;
37     static constexpr int TIMEOUT_SECONDS = 5;
38
43     bool authenticateClient();
44
49     bool processData();
50
57     bool receiveExactly(void* buffer, size_t size);
58
65     bool sendExactly(const void* buffer, size_t size);
66
72     bool waitForData(int timeoutSeconds = TIMEOUT_SECONDS);
73
81     bool receiveWithTimeout(void* buffer, size_t size, int timeoutSeconds = TIMEOUT_SECONDS);
82
90     ssize_t recvWithTimeout(void* buffer, size_t size, int timeoutSeconds = TIMEOUT_SECONDS);
91
92 public:
100     ConnectionHandler(int socket, ClientDatabase& db, Logger& log, const sockaddr_in& addr);
101
107     void handleConnection();
108 };
```

4.13 Файл DataProcessor.cpp

Реализация класса [DataProcessor](#).

```
#include "DataProcessor.h"
```

Граф включаемых заголовочных файлов для DataProcessor.cpp:



4.13.1 Подробное описание

Реализация класса [DataProcessor](#).

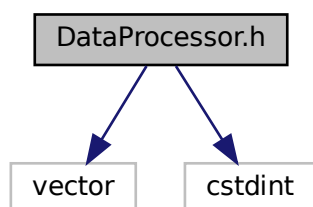
4.14 Файл DataProcessor.h

Заголовочный файл класса [DataProcessor](#) для обработки данных

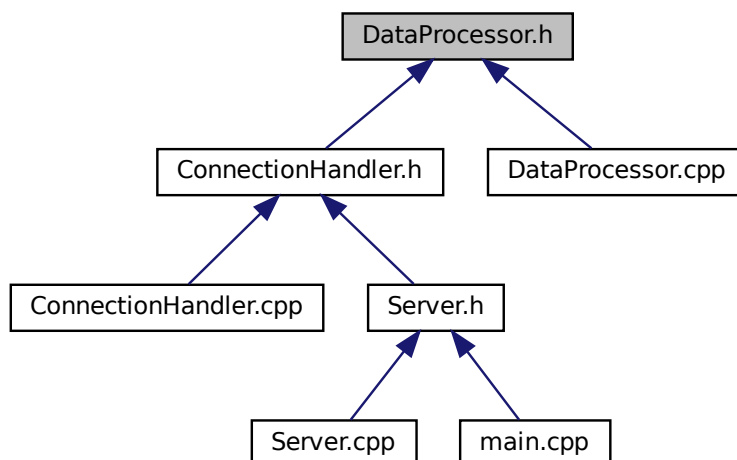
```
#include <vector>
```

```
#include <cstdint>
```

Граф включаемых заголовочных файлов для DataProcessor.h:



Граф файлов, в которые включается этот файл:



Классы

- class [DataProcessor](#)

Класс для обработки числовых данных

4.14.1 Подробное описание

Заголовочный файл класса [DataProcessor](#) для обработки данных

Содержит объявление класса [DataProcessor](#), который предоставляет методы для вычисления среднего значения и обработки переполнения.

4.15 DataProcessor.h

См. документацию.

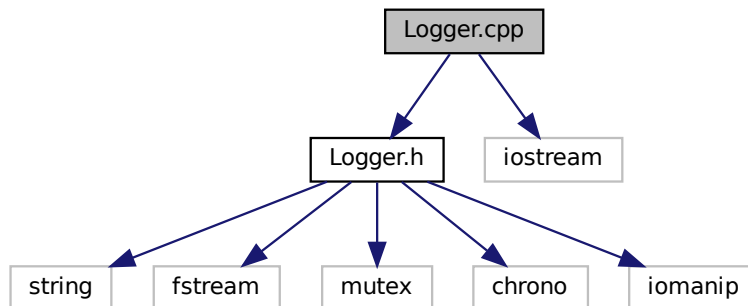
```
1
9 #pragma once
10
11 #include <vector>
12 #include <cstdint>
13
21 class DataProcessor {
22 public:
28     static uint32_t calculateAverage(const std::vector<uint32_t>& vector);
29
39     static uint32_t handleOverflow(uint64_t value);
40 };
```

4.16 Файл Logger.cpp

Реализация класса `Logger`.

```
#include "Logger.h"  
#include <iostream>
```

Граф включаемых заголовочных файлов для `Logger.cpp`:



4.16.1 Подробное описание

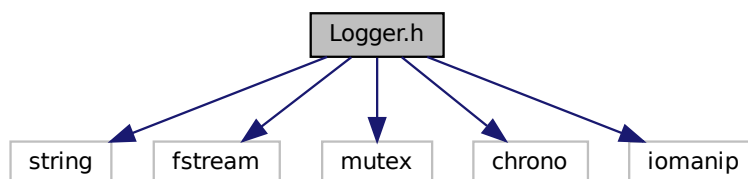
Реализация класса `Logger`.

4.17 Файл Logger.h

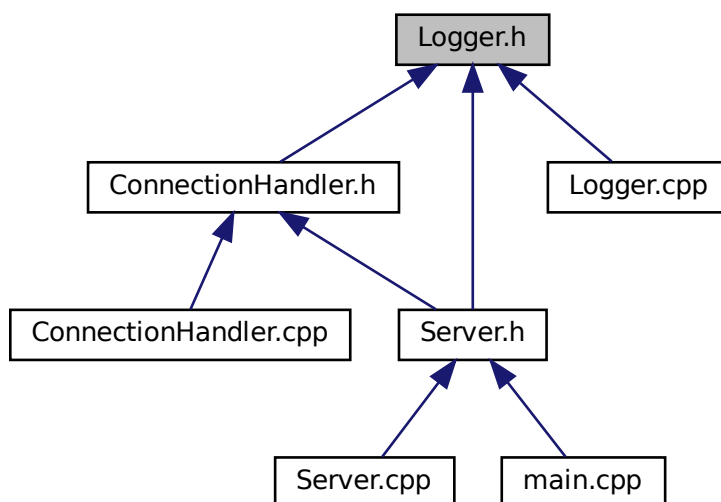
Заголовочный файл класса `Logger` для ведения журнала событий

```
#include <string>  
#include <fstream>  
#include <mutex>  
#include <chrono>  
#include <iomanip>
```

Граф включаемых заголовочных файлов для `Logger.h`:



Граф файлов, в которые включается этот файл:



Классы

- class [Logger](#)

Класс для ведения журнала событий

4.17.1 Подробное описание

Заголовочный файл класса [Logger](#) для ведения журнала событий

Содержит объявление класса [Logger](#), который предоставляет методы для записи информационных и ошибочных сообщений в файл и консоль.

4.18 Logger.h

[См. документацию.](#)

```

1
9 #pragma once
10
11 #include <string>
12 #include <fstream>
13 #include <mutex>
14 #include <chrono>
15 #include <iomanip>
16
24 class Logger {
25 private:
26     std::ofstream logFile;
27     std::mutex logMutex;
28     std::string filename;
29
34     std::string getcurrentTime();
35

```

```

36 public:
42     bool initialize(const std::string& filename);
43
47     void close();
48
54     void logError(const std::string& message, bool critical = false);
55
60     void logInfo(const std::string& message);
61 };

```

4.19 Файл main.cpp

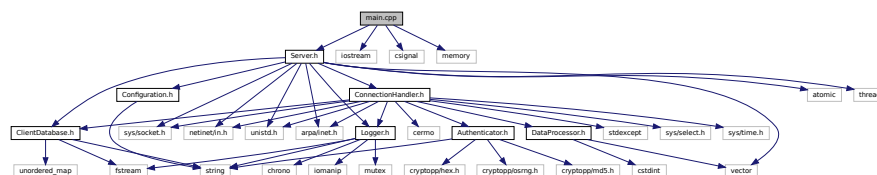
Основной файл приложения сервера

```

#include "Server.h"
#include <iostream>
#include <csignal>
#include <memory>

```

Граф включаемых заголовочных файлов для main.cpp:



Функции

- void `signalHandler` (int signal)
Обработчик сигналов операционной системы
- int `main` (int argc, char *argv[])
Точка входа приложения

Переменные

- std::unique_ptr< `Server` > server
Указатель на основной объект сервера

4.19.1 Подробное описание

Основной файл приложения сервера

Содержит точку входа приложения и обработчики сигналов. Управляет созданием и запуском основного объекта сервера.

4.19.2 Функции

4.19.2.1 main()

```

int main (
    int argc,
    char * argv[] )

```

Точка входа приложения

Аргументы

argc	Количество аргументов командной строки
argv	Массив аргументов командной строки

Возвращает

Код завершения программы (0 - успех, 1 - ошибка)

Основная функция приложения:

1. Устанавливает обработчики сигналов
2. Создает объект сервера
3. Запускает сервер
4. Обрабатывает исключения

4.19.2.2 signalHandler()

```
void signalHandler (  
    int signal )
```

Обработчик сигналов операционной системы

Аргументы

signal	Номер полученного сигнала
--------	---------------------------

Обрабатывает сигналы SIGINT (Ctrl+C) и SIGTERM для корректного завершения работы сервера.

4.20 Файл Server.cpp

Реализация класса [Server](#).

```
#include "Server.h"  
#include <iostream>  
#include <signal.h>  
#include <memory>
```


Классы

- class [Server](#)

Основной класс сервера приложения

4.21.1 Подробное описание

Заголовочный файл класса [Server](#) для управления сервером

Содержит объявление класса [Server](#), который является основным классом приложения и управляет всеми компонентами сервера.

4.22 Server.h

[См. документацию.](#)

```
1
9 #pragma once
10
11 #include <sys/socket.h>
12 #include <netinet/in.h>
13 #include <unistd.h>
14 #include <arpa/inet.h>
15 #include <atomic>
16 #include <thread>
17 #include <vector>
18 #include "Configuration.h"
19 #include "ClientDatabase.h"
20 #include "Logger.h"
21 #include "ConnectionHandler.h"
22
23 class Server {
24 private:
25     Configuration config;
26     ClientDatabase clientDB;
27     Logger logger;
28     std::atomic<bool> running;
29     int serverSocket;
30
31     bool initializeSocket();
32
33     void acceptConnections();
34
35 public:
36     Server();
37     ~Server();
38
39     bool start(int argc, char* argv[]);
40
41     void stop();
42 };
43
```


Предметный указатель

- ~Server
 - Server, [24](#)
- acceptConnections
 - Server, [24](#)
- authenticateClient
 - ConnectionHandler, [14](#)
- Authenticator, [5](#)
 - generateSalt, [5](#)
 - hashPassword, [6](#)
 - toHexString, [6](#)
 - verifyPassword, [7](#)
- Authenticator.cpp, [27](#)
- Authenticator.h, [27](#)
- calculateAverage
 - DataProcessor, [19](#)
- ClientDatabase, [8](#)
 - getPassword, [8](#)
 - loadFromFile, [9](#)
 - userExists, [9](#)
- ClientDatabase.cpp, [29](#)
- ClientDatabase.h, [30](#)
- Configuration, [10](#)
 - getClientDBFile, [11](#)
 - getLogFile, [11](#)
 - getPort, [11](#)
 - parseCommandLine, [11](#)
- Configuration.cpp, [31](#)
- Configuration.h, [31](#)
- ConnectionHandler, [12](#)
 - authenticateClient, [14](#)
 - ConnectionHandler, [14](#)
 - handleConnection, [14](#)
 - processData, [15](#)
 - receiveExactly, [15](#)
 - receiveWithTimeout, [16](#)
 - recvWithTimeout, [16](#)
 - sendExactly, [18](#)
 - waitForData, [18](#)
- ConnectionHandler.cpp, [33](#)
- ConnectionHandler.h, [33](#)
- DataProcessor, [19](#)
 - calculateAverage, [19](#)
 - handleOverflow, [20](#)
- DataProcessor.cpp, [35](#)
- DataProcessor.h, [36](#)
- generateSalt
 - Authenticator, [5](#)
- getClientDBFile
 - Configuration, [11](#)
- getCurrentTime
 - Logger, [21](#)
- getLogFile
 - Configuration, [11](#)
- getPassword
 - ClientDatabase, [8](#)
- getPort
 - Configuration, [11](#)
- handleConnection
 - ConnectionHandler, [14](#)
- handleOverflow
 - DataProcessor, [20](#)
- hashPassword
 - Authenticator, [6](#)
- initialize
 - Logger, [21](#)
- initializeSocket
 - Server, [24](#)
- loadFromFile
 - ClientDatabase, [9](#)
- logError
 - Logger, [22](#)
- Logger, [20](#)
 - getCurrentTime, [21](#)
 - initialize, [21](#)
 - logError, [22](#)
 - logInfo, [22](#)
- Logger.cpp, [38](#)
- Logger.h, [38](#)
- logInfo
 - Logger, [22](#)
- main
 - main.cpp, [40](#)
- main.cpp, [40](#)
 - main, [40](#)
 - signalHandler, [41](#)
- parseCommandLine
 - Configuration, [11](#)
- processData
 - ConnectionHandler, [15](#)
- receiveExactly

- ConnectionHandler, [15](#)
- receiveWithTimeout
 - ConnectionHandler, [16](#)
- recvWithTimeout
 - ConnectionHandler, [16](#)
- sendExactly
 - ConnectionHandler, [18](#)
- Server, [23](#)
 - ~Server, [24](#)
 - acceptConnections, [24](#)
 - initializeSocket, [24](#)
 - start, [25](#)
 - stop, [25](#)
- Server.cpp, [41](#)
- Server.h, [42](#)
- signalHandler
 - main.cpp, [41](#)
- start
 - Server, [25](#)
- stop
 - Server, [25](#)
- toHexString
 - Authenticator, [6](#)
- userExists
 - ClientDatabase, [9](#)
- verifyPassword
 - Authenticator, [7](#)
- waitForData
 - ConnectionHandler, [18](#)